Deployment Guide

Running IntelliMetrics AI locally, on a server and on free cloud hosting, with what was and was not verified

Project title	IntelliMetrics AI: A Governed Semantic Layer with TARA, an AI Assistant for Trusted Business Metrics
Document	Deployment Guide
Team members	Devi Priya, Priyanka, Akshay and Bhaskar
Institution	KL University
Academic year	2026-27
Date	26-Oct-2026 (IST)
Status	University project: academic capstone with synthetic demonstration data

Table of contents

- 1 Purpose and honesty note
- 2 Architecture to deploy
- 3 Option 1: local machine without Docker (tested)
- 4 Option 2: Docker Compose on one machine (written, not run)
- 5 Option 3: a small VPS with a domain (documented only)
- 6 Option 4: free cloud hosting services (documented only)
- 7 Environment variable reference
- 8 Migrations, backups and logs
- 9 Security checklist for a public demonstration
- 10 Troubleshooting
- 11 Known limitations of the deployment design

1 Purpose and honesty note

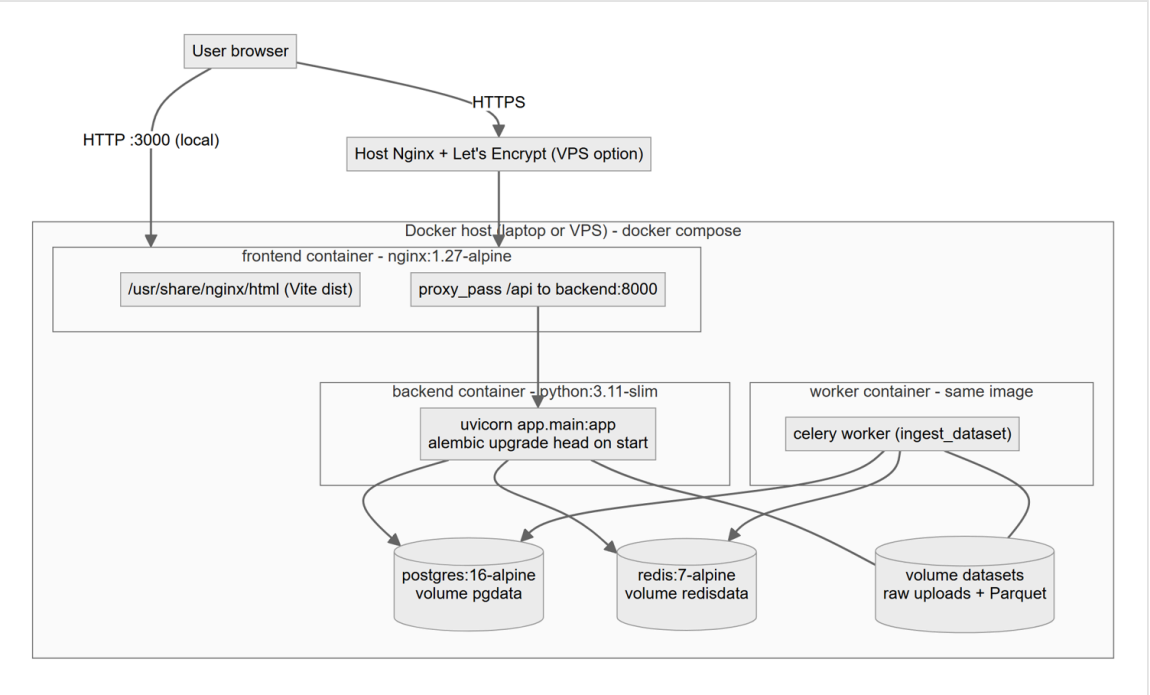
This guide explains how to deploy IntelliMetrics AI. Because it is a university project, the aim is a **cheap or free, reproducible deployment** that a student can run for a demonstration, not a hardened enterprise installation.

Each deployment path below is marked with how far it was actually exercised. Read this table first.

Path	Status in this project
Local, no Docker (SQLite + in-process worker)	**Run and tested** (all automated suites run on it)
PostgreSQL 18.2 as application database	**Verified**: Alembic migrations applied from empty, then the full back-end suite (136 tests) passed
Redis + Celery worker	**Verified**: a real worker consumed an ingestion job from a real Redis broker (job 748 ms for 62,000 rows)
PostgreSQL and MySQL as *import sources*	**Verified** against isolated local servers (PostgreSQL 18.2, MySQL 8.0.40)
Production front-end build (`vite build`)	**Run** successfully
Docker images and `docker compose up`	**Written, not run.** Docker is not installed on the development machine, so the images were never built
VPS with Nginx and Let's Encrypt	**Documented only.** No server was used; the commands follow standard practice
Free cloud hosting (Render, Railway, Vercel, Netlify)	**Documented only.** No account was created and nothing was deployed

Anything marked *documented only* should be tried first on a throw-away instance and corrected if reality differs.

2 Architecture to deploy



Deployment topology used by Docker Compose

Component	Role	Needed for a demo?
Front end (static files)	React single-page application	Yes
API (FastAPI + Uvicorn)	All business logic; serves `/api`	Yes
Database	Users, metrics, versions, conflicts, dashboards, audit. SQLite for a demo, PostgreSQL otherwise	Yes
Dataset volume	Raw uploads and Parquet files read by DuckDB	Yes (must persist and be shared by API and worker)
Redis + Celery worker	Background dataset processing	Optional: without them a thread inside the API does the job
Nginx	Serves the front end, proxies `/api`, disables buffering for streaming answers	In Compose and on a VPS

3 Option 1: local machine without Docker (tested)

See the User Guide for the step-by-step Windows commands. In short: create a virtual environment, `pip install -r backend/requirements.txt`, `python -m app.seed`, `uvicorn app.main:app --port 8000`, then `npm install` and `npm run dev` in `frontend`.

To use the production-style topology locally without Docker (this is what was verified):

```
# PostgreSQL and Redis running locally on spare ports
export DATABASE_URL="postgresql://user:pass@127.0.0.1:5544/intellimetrics"
export REDIS_URL="redis://127.0.0.1:6390/0"
export USE_CELERY=true  SECRET_KEY="a-long-random-string-of-at-least-32-characters"
cd backend
alembic upgrade head
python -m app.seed
uvicorn app.main:app --port 8000 &
celery -A app.worker.celery_app worker --pool=solo --loglevel=INFO   # on Windows use --pool=solo
```

4 Option 2: Docker Compose on one machine (written, not run)

Prerequisites: Docker Engine 24 or newer with the Compose plugin, about 3 GB of disk, and free ports 3000 and 8000.

```
cp .env.example .env
python -c "import secrets; print(secrets.token_urlsafe(48))"   # paste as SECRET_KEY
docker compose up --build -d
docker compose logs -f backend      # wait for "Application startup complete"
```

Services started: `postgres` (16), `redis` (7), `backend` (runs `alembic upgrade head`, then seeds when `AUTO_SEED=true` and the database is empty), `worker` (Celery) and `frontend` (Nginx). Open http://localhost:3000.

Smoke test:

```
curl -fsS localhost:8000/health
docker compose ps
docker compose exec backend alembic current      # expect 0002 (head)
```

Stop, keep data: `docker compose stop`. Remove containers but keep volumes: `docker compose down`. Remove everything including data: `docker compose down -v`. Reset the demonstration: `./scripts/reset-demo.sh`.

5 Option 3: a small VPS with a domain (documented only)

Suitable for a public demonstration at low cost (a small virtual machine with 2 GB of memory is enough for the demo dataset).

1. **Server**: Ubuntu 22.04 or later. Install Docker (`curl -fsSL https://get.docker.com | sh`) and open only SSH, 80 and 443 (`ufw allow OpenSSH; ufw allow 80,443/tcp; ufw enable`).
2. **DNS**: an A record for `app.example.com` pointing at the server.
3. **Configure** `.env` : `ENV=production` , strong `SECRET_KEY` , `ENCRYPTION_KEY` , `POSTGRES_PASSWORD` , `COOKIE_SECURE=true` , `AUTO_SEED=true` for a demonstration (use `false` for real data), `FRONTEND_URL=https://app.example.com` , `CORS_ORIGINS=https://app.example.com` .
4. **Do not publish internal ports.** Publish the front end only on loopback (`127.0.0.1:3000:80`) and publish no back-end port, using a `docker-compose.override.yml` .
5. **Start**: `docker compose up --build -d` .
6. **TLS**: install Nginx and Certbot on the host, proxy `/` to `127.0.0.1:3000` with `proxy_buffering off` (so streamed answers work) and `client_max_body_size 600m` , then run `certbot --nginx -d app.example.com --redirect` . Renewal is automatic through the `certbot.timer` .
7. **Verify**: sign in, check that cookies are `Secure` and `HttpOnly` , upload a file, and ask TARA a question.
8. **Update**: `git pull && docker compose up --build -d` . Migrations run automatically when the back end starts.

6 Option 4: free cloud hosting services (documented only)

A typical zero-cost arrangement for a demonstration. Free hosting sleeps when idle and limit memory and storage, so expect a cold start of tens of seconds and keep the dataset small.

Piece	Service	Notes
Front end	Vercel or Netlify (static)	Build command `npm ci && npm run build`, publish `frontend/dist`. Add a rewrite from `/api/*` to the API's address so cookies stay first-party, and a fallback from `/*` to `/index.html`
API	Render or Railway web service	Use `docker/backend.Dockerfile`. Health check `/health`. Set `COOKIE_SECURE=true`; if the front end and API are on different sites also set `COOKIE_SAMESITE=none`
Database	Managed PostgreSQL from the same provider	Set `DATABASE_URL`
Background jobs	Run in the API process (`USE_CELERY=false`)	Free hosting rarely allow a separate worker sharing disk
Files	A persistent disk mounted at `DATA_DIR`, or accept that uploads vanish on restart	**Shared-storage caveat**: the API stores uploads and the worker converts them, so both must see the same files. On platforms where a disk attaches to one service, run ingestion in the API

If the demonstration needs the seeded Indian dataset, set `AUTO_SEED=true`; the 62,000-row dataset is generated in a few seconds, so no upload is needed.

7 Environment variable reference

Variable	Purpose	Example	Required
`ENV`	`development`, `production` or `test`. Only `development` returns the password-reset link in the API response	`production`	No
`DATABASE_URL`	SQLAlchemy URL; `postgresql://` uses the psycopg 3 driver. Default: local SQLite file	`postgresql://user:pw@postgres:5432/intellimetrics`	Yes in production
`REDIS_URL`	Celery broker and result store	`redis://redis:6379/0`	With `USE_CELERY=true`
`USE_CELERY`	`true` uses Celery workers; `false` uses an in-process thread	`true`	No
`SECRET_KEY`	Signs login tokens; also derives the encryption key when `ENCRYPTION_KEY` is empty. At least 32 random characters		**Yes: change the default**
`ENCRYPTION_KEY`	Fernet key protecting stored connection passwords		Recommended
`ACCESS_TTL_MIN` / `REFRESH_TTL_DAYS`	Token lifetimes (15 minutes / 7 days)		No
`COOKIE_SECURE`	Send cookies over HTTPS only	`true`	**Yes behind HTTPS**
`COOKIE_SAMESITE`	`lax`, `strict` or `none`	`lax`	No
`CORS_ORIGINS`	Allowed browser origins (not needed when one origin serves both)	`https://app.example.com`	No
`FRONTEND_URL`	Base address used in reset links	`https://app.example.com`	In production
`DATA_DIR`	Directory for raw files and Parquet	`/data/datasets`	No

Variable	Purpose	Example	Required
`MAX_UPLOAD_MB`	Upload size limit	`500`	No
`QUERY_TIMEOUT_SECONDS` / `QUERY_ROW_LIMIT`	Safety limits on every query	`20` / `10000`	No
`RATE_LIMIT_ENABLED` / `AUTH_RATE_LIMIT`	Per-process limiter on authentication routes	`true` / `20`	No
`AI_PROVIDER`	`openai`, `gemini` or `none` (offline TARA)	`none`	No
`OPENAI_API_KEY`, `OPENAI_MODEL`, `GEMINI_API_KEY`, `GEMINI_MODEL`	Live AI settings, backend only		Only for live AI
`AI_TIMEOUT_SECONDS`	AI call timeout; on failure TARA falls back to offline mode	`25`	No
`AUTO_SEED`	Seed demonstration data when the database is empty	`true`	No
`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`	Compose database bootstrap		Compose only
`FRONTEND_PORT`, `BACKEND_PORT`	Published host ports	`3000`, `8000`	Compose only
`WEB_CONCURRENCY`, `CELERY_CONCURRENCY`	Uvicorn workers and Celery processes	`2`, `2`	No

8 Migrations, backups and logs

Migrations use Alembic. `0001` creates every table including the partial unique index that guarantees one live definition per metric; `0002` installs the database trigger that makes `metric_versions` immutable. Both were applied to an empty PostgreSQL 18.2 database in this project.

```
alembic upgrade head
alembic current
alembic revision --autogenerate -m "describe change"
alembic downgrade -1
```

Backups (documented, not exercised): `pg_dump -Fc` daily to a file outside the server, `pg_restore --clean --if-exists` to restore, and a compressed copy of the `datasets` volume. SQLite deployments: copy `backend/storage` while the application is stopped. Test a restore before relying on a backup.

Logs are structured JSON; every line carries a `request_id`, and error responses echo it in `X-Request-ID` so a user's report can be matched to a log line and an audit-log row.

9 Security checklist for a public demonstration

- [ ] `SECRET_KEY` is unique and at least 32 random characters; `ENCRYPTION_KEY` is set and backed up (losing it makes stored connection passwords unreadable).
- [ ] `ENV=production`; demo accounts are acceptable only for a demonstration with synthetic data. **Their passwords are published in this documentation**, so never place real data in a deployment that still has them.
- [ ] `COOKIE_SECURE=true` and HTTPS everywhere.
- [ ] PostgreSQL and Redis are not exposed to the internet.
- [ ] `CORS_ORIGINS` lists only real origins (never `*`).
- [ ] The per-process rate limiter is supplemented by Nginx `limit_req` on `/api/auth/` if more than one API process runs.
- [ ] Any external database connection uses a read-only database role.
- [ ] AI keys exist only in the back-end environment, never in the front-end build.
- [ ] Image sources `images.unsplash.com` and `images.pexels.com` are allowed by any content-security policy, or the gradient fallbacks are accepted.

10 Troubleshooting

Symptom	Likely cause	Fix
Login succeeds but the next request is 401	`COOKIE_SECURE=true` over plain HTTP, or front end and API on different sites with `SameSite=lax`	Use HTTPS; for cross-site hosting set `COOKIE_SAMESITE=none` and `COOKIE_SECURE=true`, or proxy `/api` through the front end's origin
`403 csrf`	A custom client sent cookies without `X-Requested-With: IntelliMetrics`	Send the header, or use a bearer token
CORS error in the browser console	Origin not listed	Add the exact origin, restart the API
API restarts, log says the database is not ready	PostgreSQL still starting or wrong `DATABASE_URL`	The entrypoint retries; check credentials
Dataset stays *pending*	`USE_CELERY=true` but no worker or Redis	Start the worker, check `REDIS_URL`
Dataset fails with "file not found"	API and worker do not share `DATA_DIR`	Mount one volume in both, or set `USE_CELERY=false`
Upload fails with 413	`MAX_UPLOAD_MB` or Nginx `client_max_body_size` too small	Raise both
Ask TARA text appears all at once	A proxy buffers streamed responses	Set `proxy_buffering off`
TARA shows "offline deterministic mode"	No AI key	Set `AI_PROVIDER` and the key in the back-end environment
`Query exceeded the 20 s time limit`	Heavy query	Narrow the date range or raise `QUERY_TIMEOUT_SECONDS`
"Function 'x' is not allowed"	The formula grammar is a whitelist	Use the documented aggregate and scalar functions
"already the published definition"	Only one live definition per metric	Resolve the conflict or deprecate the other
Reset link not shown	`ENV=production` by design	Wire an e-mail provider; the project bundles none
`database is locked` when reseeding on Windows	The API still has the SQLite file open	Stop the API first
`bind: address already in use`	Port taken	Change `FRONTEND_PORT` or `BACKEND_PORT`

11 Known limitations of the deployment design

1. **No e-mail provider is bundled**, so password-reset links are shown only in development.
2. **The rate limiter is per process**; with several API workers the effective limit is multiplied.
3. **Container images were never built** in this project; the first `docker compose up --build` may reveal a packaging mistake that the tests could not. Also note that Compose pins PostgreSQL 16 whereas the verification used PostgreSQL 18.2; no PostgreSQL 16 server was run.
4. **Shared storage**: API and worker must see the same dataset directory; this limits some free cloud hosting.
5. **Single-region, single-database** design; there is no high-availability configuration.